

Segunda práctica

HUGO TONATIUH CARRILLO BONILLA

320026379



Parte 1: Editor de texto.

Actividad 1.1

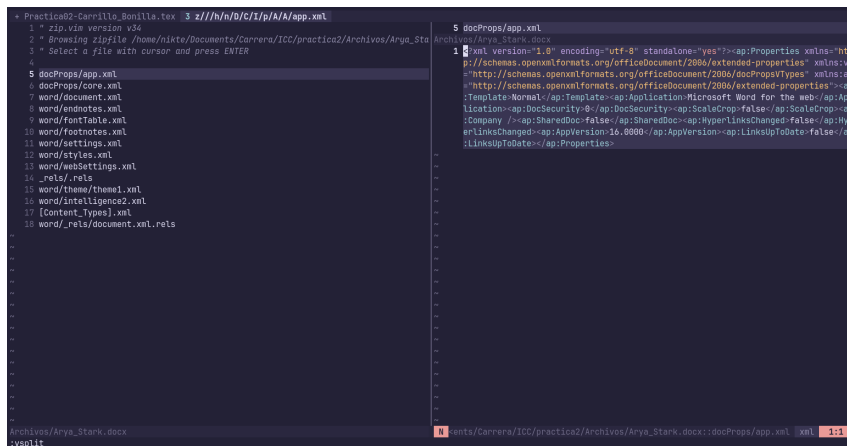
Yo utilizo el editor de texto Neovim. Lo utilizo por su simplicidad y gran capacidad. Lo utilizaré a lo largo de este curso por tres razones principales:

- Familiaridad: lo conozco desde hace años.
- Potencia: el lenguaje de programación de Java, al ser orientado a objetos, se beneficia ampliamente de las capacidades de procesamiento textual amplio de la familia de editores vi.
- Adaptabilidad: Existe un gran ambiente de extensiones y herramientas para Java. Además, es completamente gratis y de código abierto. Mientras que *SublimeText* es de código cerrado y freemium.

```
1 /*  
2  * Comentario multilinea para que vean que es así propia captura y no la saqué de Internet. -Niké  
3  */  
4 public class HolaMundo {  
5     public static void main(String args[]) {  
6         System.out.println("Hola mundo.")  
7     }  
8 }
```

Actividad 1.2

- a Pude abrir sin absolutamente ningún problema los archivos `.txt` y `.java`. Con los archivos `.docx` y `.xlsx` sucedió algo muy interesante: descubrí que *Vim* tiene un plugin interno que te permite interactuar con archivos comprimidos y esto incluye a los xml. Entonces tuve acceso a un pseudoménú que después me permitía acceder individualmente a los archivos xml que estaban subcontenidos en cada archivo propietario de Windows.



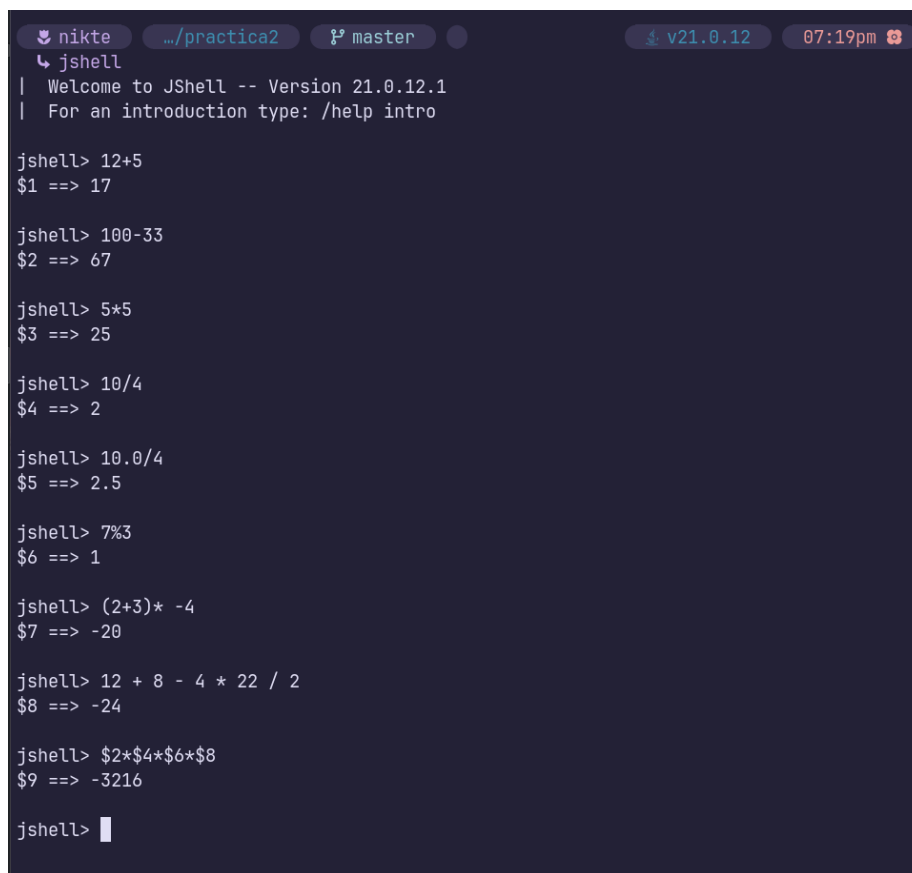
```
Practica2-Cervilla_Bonilla.docx 3 z://h/n/D/C/1/p/h/n/app.xml  
1 * zip file version x86  
2 * Browsing zipfile /home/nike/Documents/Carrera/ICC/practica2/Archivos/Anyo_Sta  
3 * Select a file with cursor and press ENTER  
4  
5 docProps/app.xml  
6 docProps/core.xml  
7 word/document.xml  
8 word/endnotes.xml  
9 word/fontTable.xml  
10 word/footnotes.xml  
11 word/settings.xml  
12 word/styles.xml  
13 word/webSettings.xml  
14 _rels/.rels  
15 word/theme/theme1.xml  
16 word/intelligence2.xml  
17 [Content_Types].xml  
18 word/_rels/document.xml.rels  
19  
20 Archives/Anyo_Stark.docx  
:vsplit
```

Finalmente, con los archivos `.pdf` y `.png` solo ví un montón de caracteres aleatorios sin ton ni son.

- b La razón por la que puedo abrir perfectamente los archivos de texto y de Java es precisamente porque son archivos de texto pensados para ser mostrados en un editor de texto plano, los archivos propietarios de Windows en realidad son empaquetadores elegantes para muchos archivos del tipo xml. Finalmente, los archivos de **.pdf** y **.png** son archivos principalmente visuales que codifican su información a través de ecuaciones o mapas.

Parte 2: JShell

Actividad 2.1



```
nikte .../practica2 master v21.0.12 07:19pm
jshell
| Welcome to JShell -- Version 21.0.12.1
| For an introduction type: /help intro

jshell> 12+5
$1 ==> 17

jshell> 100-33
$2 ==> 67

jshell> 5*5
$3 ==> 25

jshell> 10/4
$4 ==> 2

jshell> 10.0/4
$5 ==> 2.5

jshell> 7%3
$6 ==> 1

jshell> (2+3)* -4
$7 ==> -20

jshell> 12 + 8 - 4 * 22 / 2
$8 ==> -24

jshell> $2*$4*$6*$8
$9 ==> -3216

jshell> 
```

Mi laptop soñada es la *Lenovo ThinkPad P16s* sus características son:

- Procesador Intel i9
- 32 GB de RAM
- 1 TB de memoria SSD

Para calcular su precio sin IVA, guardé su precio en total en una variable, guardé la diferencia entre el 100% y el porcentaje sin IVA en otra variable, y realicé la siguiente operación con estas dos variables:

$$\text{Precio} \cdot \text{Diferencia} \times 10^{-2}$$

```
nikte .../practica2 master ?(1) v21.0.12 07:23pm
└─ jshell
| Welcome to JShell -- Version 21.0.12.1
| For an introduction type: /help intro

jshell> 104807.23
$1 ==> 104807.23

jshell> 100-16
$2 ==> 84

jshell> $1*($2/100.0)
$3 ==> 88038.0732

jshell> 
```

Actividad 2.2

```
🌸 nikte ~  
↳ jshell  
| Welcome to JShell -- Version 21.0.12.1  
| For an introduction type: /help intro  
  
jshell> 0b1010  
$1 ==> 10  
  
jshell> 012  
$2 ==> 10  
  
jshell> 0xA  
$3 ==> 10  
  
jshell> Integer.toBinaryString(10)  
$4 ==> "1010"  
  
jshell> Integer.toOctalString(10)  
$5 ==> "12"  
  
jshell> Integer.toHexString(10)  
$6 ==> "a"  
  
jshell> █
```

Decimal a binario.

$$\begin{array}{r} 6 \\ 1. \quad 2 \overline{)13} \quad 2 \overline{)6} \quad 2 \overline{)3} \quad 2 \overline{)1} \\ \underline{12} \quad \underline{6} \quad \underline{2} \\ 1 \quad 0 \quad 1 \end{array}$$

$$\begin{array}{r} 21 \quad 10 \quad 5 \quad 2 \quad 1 \quad 0 \\ 2. \quad 2 \overline{)42} \quad 2 \overline{)21} \quad 2 \overline{)10} \quad 2 \overline{)5} \quad 2 \overline{)2} \quad 2 \overline{)1} \\ \underline{4} \quad \underline{2} \quad \underline{10} \quad \underline{4} \quad \underline{2} \\ \underline{02} \quad \underline{01} \quad \underline{0} \quad \underline{1} \quad \underline{0} \\ 2 \\ \underline{0} \end{array}$$

Binario a decimal.

1.

$$1001001_2 = (1 \cdot 2^0) + (0 \cdot 2^1) + (0 \cdot 2^2) + (1 \cdot 2^3) + (0 \cdot 2^4) + (0 \cdot 2^5) + (1 \cdot 2^6) = 73_{10}$$

2.

$$11011011_2 = (1 \cdot 2^0) + (1 \cdot 2^1) + (0 \cdot 2^2) + (1 \cdot 2^3) + (1 \cdot 2^4) + (0 \cdot 2^5) + (1 \cdot 2^6) + (1 \cdot 2^7) = 219_{10}$$

```
nikte ~
↳ jshell
| Welcome to JShell -- Version 21.0.12.1
| For an introduction type: /help intro

jshell> Integer.toBinaryString(13)
$1 ==> "1101"

jshell> Integer.toBinaryString(42)
$2 ==> "101010"

jshell> 0b1001001
$3 ==> 73

jshell> 0b11011011
$4 ==> 219

jshell> █
```

Parte 3: Autopsia de un programa

Actividad 3.1

Así me corrió el Hola Mundo:

```
nikte ~/practica2 P master ?(1) v21.0.12 08:19pm
↳ javac Bienvenida.java

nikte ~/practica2 P master ?(2) v21.0.12 08:20pm
↳ java Bienvenida
Hola mundo!

nikte ~/practica2 P master ?(2) v21.0.12 08:20pm
↳
```

Para el ejercicio 4 modifiqué así el programa:

```

1 public class Bienvenida {
2 »     public static void main(String[] args) {
3 »         System.out.println("Hugo Tonatiuh Carrillo Bonilla");
4 »         System.out.println("320026379");
5 »         System.out.println("No considero tener expertiz en nada.");
6 »     }
7 }

```

Mientras que para el ejercicio 5 lo modifiqué así:

```

1 public class Bienvenida {
2 »     public static void main(String[] args) {
3 »         System.out.println("Hugo Tonatiuh Carrillo Bonilla \n 320026379 \n No considero tener expertiz en nada.");
4 »     }
5 }

```

Compiló así:

```

$ nikto -u /practica2 -P master -v21.0.12 02:17pm
$ javac Bienvenida.java
$ nikto -u /practica2 -P master -v21.0.12 02:17pm
$ java Bienvenida
Hugo Tonatiuh Carrillo Bonilla
320026379
No considero tener expertiz en nada.
$ nikto -u /practica2 -P master -v21.0.12 02:17pm

```

Actividad 3.2

Los errores que se encontraron al principio fueron:

1. Línea 5. Código mal escrito. Faltaba cerrar un paréntesis. **Errores.java:5: error: ')' or ',' expected.** Se corrige añadiendo el paréntesis final **antes** del punto y coma.
2. Línea 6. Código mal escrito. Faltaba cerrar una string. **Errores.java:6: error: unclosed string literal.** Se corrige añadiendo las comillas finales.
3. Línea 12. Código mal escrito. No se termina adecuadamente una expresión. **Errores.java:12: error: ';' expected.** Se corrige añadiendo punto y coma al final.
4. Línea 7. Código sin sentido. El compilador piensa que queremos restar strings. **Errores.java:7: error: bad operand types for binary operator '-'**. Ponemos la resta entre paréntesis.
5. Línea 10. Código sin sentido. Le dimos una string a una función que solo acepta enteros. **Errores.java:10: error: incompatible types: String cannot be converted to int.** Le quitamos las comillas al argumento dentro de la función.
6. Línea 8. Código con resultado incorrecto. Dividimos dos enteros pero queremos un float. No arroja errores porque hace lo que le pedimos, no supimos expresar lo que queríamos. Cambiamos alguno de los valores divididos a un float.

7. Línea 9. Código con resultado incorrecto. Nuestro orden de operaciones es incorrecta. No arroja errores porque hace lo que le pedimos, no supimos expresar lo que queríamos. Insertamos paréntesis para que el orden de operaciones sea explícito.
8. Línea 11. Código con resultado incorrecto. Exigimos que la función `Integer.parseInt` nos devuelva un número en base 8, pero queremos que nos devuelva 39, y para eso necesitamos que traduzca un número en base 5. No arroja errores porque hace lo que le pedimos, no supimos expresar lo que queríamos. Cambiamos el segundo argumento de la función por 5.



```
nikte /practice2 P master
javac Errores.java

nikte /practice2 P master
java Errores
== Ficha de compra ==
Producto: Laptop para la carrera
Precio con descuento: 12000
Plazo en años: 1.5
Pago mensual: 666.6666666666666
Folio en binario: 11010
Clave descifrada: 39
Fin de la ficha
```

Conclusión

En esta práctica me familiaricé con los errores y logs del compilador de Java. Me salió un error que no me pareció prudente comentar pero es apropiado señalar aquí: la *Java Virtual Machine* es muy sensible con que le demos la clase correcta, y para eso nos debemos de encontrar en el mismo directorio en el que se encuentra nuestro archivo `.class`. Eso me pareció interesante, y gracias a este error aprendí un poco sobre cómo la JVM corre e inicializa los programas compilados. Escuché por primera vez los términos `ClassLoader`, `ObjectLoader`, `ClassPath`, *JAR Hell* entre otros. Estos son conceptos por los que me emociona mucho empezar a aprender.